Project 01 — ANN Binary Classification

Level: Basic | **Dataset:** Heart Disease UCI | **Framework:** PyTorch

---

Objective

Build a feedforward neural network to predict heart disease (0/1) from clinical features.
Cover: weight initialization, BatchNorm, Dropout, training loop, metrics, Streamlit inference.

Project Structure

```
01_ann_binary_classification/
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_preprocessing.ipynb
│   └── 03_train_evaluate.ipynb
├── data/raw/heart.csv
├── data/processed/
├── models/model.pkl
├── path_utils.py
├── dashboard_core.py
└── app.py
```

Notebook 01 — EDA (`01_eda.ipynb`)

STOP 1 — Dataset Overview

- Load `heart.csv` using pandas
- Print `.shape`, `.dtypes`, `.describe()`
- Print class distribution of target column
- **Agent stops here. Explain to user:**
  - What binary classification means
  - What each feature represents (age, chol, thalach, etc.)
  - What class imbalance is and why it matters
- Wait for user confirmation before continuing

STOP 2 — Missing Values & Distributions

- Check `.isnull().sum()`
- Plot histogram for each numerical feature using matplotlib
- Plot countplot for categorical features
- **Agent stops here. Explain:**
  - Why we visualize distributions before training
  - Difference between numerical and categorical features in neural nets
- Wait for confirmation

STOP 3 — Correlation Analysis

- Compute `df.corr()` on numeric columns
- Plot heatmap using seaborn
- Identify top 3 features correlated with target
- **Agent stops here. Explain:**
  - What correlation tells us and what it doesn't
  - Why correlated features matter for ANNs
- Wait for confirmation

Notebook 02 — Preprocessing (`02_preprocessing.ipynb`)

STOP 4 — Train/Test Split

- Use `train_test_split` with `stratify=y`, `test_size=0.2`, `random_state=42`
- Print class distribution in both splits
- **Agent stops here. Explain:**
  - Why we stratify on the target
  - Why random state matters for reproducibility
- Wait for confirmation

STOP 5 — Feature Scaling

- Apply `StandardScaler` on numerical features only
- Fit on train, transform on both train and test (no data leakage)
- Save scaler to `data/processed/scaler.pkl`
- **Agent stops here. Explain:**
  - Why neural networks need scaled inputs
  - What data leakage is and how fitting on test causes it
  - What StandardScaler does mathematically (mean=0, std=1)
- Wait for confirmation

STOP 6 — Convert to PyTorch Tensors

- Convert X_train, X_test, y_train, y_test to `torch.FloatTensor`
- Create `TensorDataset` and `DataLoader` with `batch_size=32`
- **Agent stops here. Explain:**
  - What a DataLoader does and why batching matters
  - Difference between FloatTensor and LongTensor and when to use each
- Wait for confirmation

Notebook 03 — Train & Evaluate (`03_train_evaluate.ipynb`)

STOP 7 — Model Architecture

Define model class:

```
Input(13) → Linear(64) → BatchNorm1d → ReLU → Dropout(0.3)
          → Linear(32) → BatchNorm1d → ReLU → Dropout(0.3)
          → Linear(1)  → Sigmoid
```

- **Agent stops here. Explain:**
  - What BatchNorm1d does: normalizes activations per batch, stabilizes training
  - What Dropout does: randomly zeros neurons, prevents overfitting
  - Why Sigmoid at output for binary classification
  - Difference between model.train() and model.eval() mode
- Wait for confirmation

STOP 8 — Weight Initialization

- Define `init_weights(m)` function
- Apply `kaiming_uniform_` for Linear layers, `constant_` for bias
- Use `model.apply(init_weights)`
- **Agent stops here. Explain:**
  - Why random initialization matters (symmetry breaking)
  - What Kaiming init is: designed for ReLU activations
  - What happens with all-zero or all-same initialization
- Wait for confirmation

STOP 9 — Loss Function & Optimizer

- Use `BCELoss()` as criterion
- Use `Adam(model.parameters(), lr=0.001)`
- **Agent stops here. Explain:**
  - Why BCELoss for binary classification
  - What Adam does vs SGD (adaptive learning rates)
  - What lr=0.001 means and how to choose it
- Wait for confirmation

STOP 10 — Training Loop

Write full training loop:

- Forward pass → loss → backward → optimizer step
- Track train loss and accuracy per epoch
- Run for 100 epochs
- **Agent stops here. Explain:**
  - The four steps of every training loop (zero_grad, forward, loss, backward, step)
  - Why we call zero_grad() before each backward pass
  - What happens if we don't reset gradients
- Wait for confirmation

STOP 11 — Validation Loop

- Set `model.eval()`, use `torch.no_grad()`
- Compute val loss and val accuracy per epoch
- Plot train vs val loss curve
- **Agent stops here. Explain:**
  - Why model.eval() is necessary (disables Dropout and BatchNorm train mode)
  - Why torch.no_grad() saves memory
  - How to read a loss curve: overfitting vs underfitting signals
- Wait for confirmation

STOP 12 — Evaluation Metrics

- Compute on test set: Accuracy, Precision, Recall, F1, ROC-AUC
- Plot confusion matrix
- Plot ROC curve
- **Agent stops here. Explain:**
  - Why accuracy alone is misleading for imbalanced data
  - What Precision/Recall tradeoff means
  - What AUC-ROC tells us (model's ability to rank positives above negatives)
- Wait for confirmation

STOP 13 — Save Model

- Save `model.state_dict()` to `models/model.pkl`
- Save scaler from preprocessing step
- Write `predict(x)` function that loads model + scaler and returns probability
- **Agent stops here. Explain:**
  - Difference between saving state_dict vs full model
  - Why we always save the scaler with the model
- Wait for confirmation

`path_utils.py`

- Define `ROOT`, `DATA_RAW`, `DATA_PROCESSED`, `MODELS` paths using `pathlib.Path`
- Single import used by all notebooks and app.py

`dashboard_core.py`

Functions (no Streamlit imports here):

- `load_model_and_scaler()` → returns model, scaler
- `predict_proba(features_dict)` → returns float probability
- `get_training_curves()` → load saved loss/acc history, return as dict
- `get_metrics()` → return test accuracy, F1, AUC from saved results.json

---

`app.py` — Streamlit (~80 lines)

Sections:

1. Sidebar: input sliders for each of 13 features
2. Main: "Predict" button → show probability bar + label
3. Tab 1: Training curves (loss + accuracy)
4. Tab 2: Confusion matrix + ROC curve (saved images)
5. Tab 3: Model architecture summary (text)

No feature engineering in app — raw input → scaler → model → output only.

Key Concepts Covered

- Binary cross entropy loss
- BatchNorm1d mechanics
- Dropout regularization
- Kaiming weight initialization
- Adam optimizer
- train/eval mode difference
- ROC-AUC interpretation
- DataLoader batching